

二分与三分

二分法，在一个单调有序的集合或函数中查找一个解，每次分为左右两部分，判断解在哪个部分中并调整上下界，直到找到目标元素，每次二分后都将舍弃一半的查找空间，因此效率很高。

1、整数定义域上的二分

```
int bs(int L,int R)
{
    int L=1, R=n, mid, ans;
    while(L<=R)
    {
        mid=L+(R-L)>>1;
        if(check(mid))
        {
            ans=mid;
            L=mid+1;
        }
        else R=mid-1;
    }
    return ans;
}
```

2、实数定义域上的二分

```
int bs(int L,int R)          //dlt=0.001 (根据题目要求决定精度)
{
    double mid;;
    while(fabs(R-L)>dlt)
    {
        mid=L+(R-L)/2.0;
        if(check(mid))
        {
            R=mid;
        }
        else L=mid;
    }
    return L
}
```

综上所述，如果我们指定二分的次数 t ，那么对于初始的求解区间长度 L ，算法结束后 $R-L$ 的值会是 $L/2^t$ ，根据这个值判断精度是否达到我们的要求即可。

二分法常见模型

1、二分答案

最小值最大（或最大值最小）问题，这类双值问题常常选用二分法求解，也就是确定答案后，配合贪心、DP等其他算法检验这个答案是否合理，将最优化问题转化为判定性问题。

2、二分查找

用具有单调性的布尔表达式求解分界点，比如在有序数列中求数字 x 的排名。

3、代替三分

有时，对于一些单峰函数，我们可以用二分导函数的方法求解函数极值，这时通常将函数的定义域定义为整数域求解比较方便，此时 dx 可以直接取整数1。

例如：愤怒的牛

农夫约翰建造了一座有 n 间牛舍的小屋，牛舍排在一条直线上，第 i 间牛舍在 x_i 的位置，但是约翰的 m 头牛对小屋很不满意，因此经常互相攻击。约翰为了防止牛之间互相伤害，因此决定把每头牛都放在离其它牛尽可能远的牛舍。也就是要最大化最近的两头牛之间的距离。

牛们并不喜欢这种布局，而且几头牛放在一个隔间里，它们就要发生争斗。为了不让牛互相伤害。John决定自己给牛分配隔间，使任意两头牛之间的最小距离尽可能的大，那么，这个最大的最小距离是多少呢？

输入格式

第一行用空格分隔的两个整数 n 和 m ；

第二行为 n 个用空格隔开的整数，表示位置 x_i 。

输出格式

样例输入输出仅一个整数，表示最大的最小距离值。

5 3

1 2 8 4 9

样例输出

3

样例解释

把牛放在第1,4,8间，这样最小距离是3。

例如：Best Cow Fences

给定一个长度为 n 的非负整数序列 A ，求一个平均数最大的，长度不小于 L 的子序列。

输入格式

第一行用空格分隔的两个整数 n 和 L ；

第二行为 n 个用空格隔开的非负整数，表示 A_i 。

输出格式

输出一个整数，表示答案的 1000 倍。（不用四舍五入，直接输出）。

样例输入

10 6

6 4 2 10 3 8 5 9 4 1

样例输出

6500

三分

三分法适用于求解凸性函数的极值问题，二次函数就是一个典型的单峰函数。

三分法和二分法一样，它会不断缩小答案所在的求解区间。二分法缩小区间利用的原理是函数的单调性，而三分法利用的则是函数的单峰性。

设当前求解的区间为 $[L, R]$ ，令 $m_1 = L + (R - L) / 3$ ， $m_2 = R - (R - L) / 3$ ，接着我们计算这两个点的函数值 $f(m_1)$ ， $f(m_2)$ 之后我们将两点中函数值更优的那个点称为好点（若计算最大值，则 f 更大的那个点就为好点，计算最小值同理），而函数值较差的那个点成为坏点。

我们可以证明，最优点与好点会与坏点同侧。如下图所示， $f(m_1) > f(m_2)$ ，则 m_1 是好点，而 m_2 是坏点，因此最后的最优点会与 m_1 一起在 m_2 的左侧，即我们的求解区间由 $[L, R]$ 变为了 $[L, m_2]$ 。因此根据这个结论我们可以不停缩小求解区间，直至得出近似解。



三分

与二分一样，我们可以指定三分的次数，或是根据 $R-L$ 的值来终止算法。

以求上凸峰函数的最大值为例，三分的参考程序为：

```
double L=0, R=1e9 ;  
while(R-L>=1e-3)  
{  
    double m1=L+(R-L)/3, m2=R-(R-L)/3;  
    if(f(m1)<f(m2)) L=m1;  
    else R=m2;  
}
```



注意，我们在介绍单峰函数时特别强调了“严格”单调性。若在三分过程中遇到 $f(m1)=f(m2)$ ，当函数严格单调时，令 $L=m1$ 或 $R=m2$ 均可。如果函数不是严格单调，即在函数中存在一段函数值相等的区间，我们将无法判断定义域的左右边界应该如何缩小，三分法就不再适用。

例如：曲线

明明做作业的时候遇到了n个二次函数 $S_i(x) = ax^2 + bx + c$ ，他突发奇想设计了一个新的函数 $F(x) = \max(S_i(x))$ ， $i = 1 \dots n$ 。

明明现在想求这个函数在 $[0, 1000]$ 的最小值，要求精确到小数点后四位四舍五入。

输入数据

输入包含T 组数据（ $T < 10$ ），每组第一行一个整数 $n(n \leq 10000)$ ，之后n行，每行3个整数a，b，c，用来表示每个二次函数的3个系数，注意二次函数有可能退化成一次。

输出数据

每组数据一个输出，表示新函数 $F(x)$ 的在区间 $[0, 1000]$ 上的最小值。精确到小数点后四位，四舍五入。

样例输入

```
2
1
2 0 0
2
2 0 0
2 -4 2
```

样例输出

```
0.0000
0.5000
```

